

Reflections — Local Mock to Real Supabase Migration Runbook

Prepared for Kiran

June 19, 2026

Contents

How to use this document	1
1. Current architecture, in brief	2
2. Target Supabase project setup	3
3. Database schema (run as one script in the Supabase SQL Editor)	3
4. Auth strategy	6
5. Code changes, file by file	6
5.1 src/lib/supabase.ts (browser client)	6
5.2 src/lib/supabase-server.ts (server client)	6
5.3 New server route for school account creation	7
5.4 Everything else under src/app — no changes expected	8
5.5 Files to delete once the migration is verified working	8
6. The password_plain decision	8
7. Row Level Security	9
8. Storage policy for the rulebooks bucket	10
9. Environment variables	10
10. Migration checklist, in order	11
11. Reference: current table shapes (as actually used, not just as typed)	11

How to use this document

This is a self-contained runbook for migrating the **Reflections** Next.js app from its current local-only mock database to a real Supabase project. It was written so that it can be handed back to Claude in a future session with no other context — every fact needed to execute the migration is in here: table schemas, exact file paths, exact line-level code changes, and the reasoning behind each decision.

If you (Kiran) are reading this to do the migration yourself, follow the checklist in the last section in order. If you're handing this PDF to Claude in a new conversation, the right opening

prompt is: “Here’s the migration runbook I generated earlier. Execute the checklist against the actual codebase, starting from the Supabase project setup.”

Nothing in this document needs to be re-derived — it was produced by reading the actual source files (local-engine.ts, local-store.server.ts, mock-client.server.ts, mock-client.browser.ts, mock-query.ts, mock-data.ts, types.ts, every page under src/app, and the /api/local/* routes) line by line, not guessed from convention.

1. Current architecture, in brief

The app currently runs in **local mode**: there is no real Supabase project connected anywhere. src/lib/supabase.ts (browser) and src/lib/supabase-server.ts (server) both unconditionally return a hand-built mock client instead of a real @supabase/supabase-js client:

```
// src/lib/supabase.ts
export function createClient() {
  return createMockBrowserClient() as any
}

// src/lib/supabase-server.ts
export function createClient() {
  const cookieStore = cookies()
  const sessionCookie = cookieStore.get(SESSION_COOKIE)?.value
  return createMockServerClient(sessionCookie) as any
}
```

The mock client (src/lib/mock-client.server.ts, mock-client.browser.ts) deliberately mirrors the real Supabase JS SDK’s surface: auth.getUser(), auth.signInWithPassword(), auth.signOut(), auth.signUp(), and a chainable from(table) query builder (mock-query.ts) supporting .select(), .eq(), .neq(), .not(), .in(), .order(), .limit(), .single(), .insert(), .update(), .upsert(), .delete(). This was a deliberate design choice and it pays off now: **most page-level code (supabase.from('events').select(...) etc.) needs zero changes** when swapped to a real client, because the chained method calls already match the real SDK 1:1.

Data lives in a single JSON file at %APPDATA%\reflections-local\reflections-db.json (chosen specifically to sit outside the OneDrive-synced project folder and avoid sync corruption). Three Next.js route handlers proxy browser reads/writes to that file, gated by an env flag:

- src/app/api/local/query/route.ts — runs executeQuery() against the JSON file
- src/app/api/local/auth/route.ts — sign-in/sign-up against the JSON file, sets the session cookie
- src/app/api/local/debug/route.ts — dumps file contents for debugging

All three check process.env.NEXT_PUBLIC_LOCAL_MODE === 'true' and 403 otherwise. This flag is currently true in .env.local.

Auth today is plaintext. local-store.server.ts’s signInUser() does store.users.find(u => u.email === email && u.password === password) — a direct string comparison against a password stored in clear text in the JSON file. This is fine for a local mock; it is the single most important thing to *not* carry forward as-is.

Roles and identity. Each mock user has {id, email, password, role: 'school'|'club_member'|'final_year_slot_number?}. The mock’s mapAuthUser() reshapes this into {id, email, user_metadata: {role, slot_number}} — which is exactly the shape of real Supabase Auth’s user_metadata.

This was clearly designed to map onto Supabase Auth directly: role and slot number belong in user_metadata (set via options.data on sign-up), not a separate hand-rolled users table.

2. Target Supabase project setup

1. Create a project at supabase.com (or self-host). Note the **Project URL** and **anon public key** from Project Settings → API. Also note the **service_role key** — this is secret and must never reach the browser.
 2. In the SQL Editor, run the schema in Section 3.
 3. In Authentication → Providers, leave Email enabled. Turn **off** “Confirm email” for now (the app currently signs users in immediately after creation; you can turn confirmation back on later and add an email-confirmation step to the school-creation flow if you want it).
 4. In Storage, create a bucket named rulebooks (Section 3.9) — this backs the file upload in admin/events/page.tsx.
-

3. Database schema (run as one script in the Supabase SQL Editor)

Design notes baked into this schema, all derived directly from how the app actually reads/writes these fields today (not just from field names):

- scores (on marks) is read and written by admin/marks/page.tsx as a **plain JS array** — row.scores ?? [], .slice(), .map() — never JSON.stringify/JSON.parse. It must be a native array type in Postgres. jsonb is used below because it accepts a plain JS array directly from supabase-js with zero code changes; a real numeric[] column would also work but needs array literal handling on insert.
- winners_json (on results) and assigned_members (on events) **are** manually JSON.stringify'd on write and JSON.parse'd on read in six call sites across the app (full list in Section 5.3). To avoid touching all six sites, both columns are kept as plain text here — Postgres just stores the JSON string exactly as today. You can migrate these to real jsonb later as a cleanup; it is not required for the migration to work.
- settings is a singleton (always exactly one row, id = 1, mutated via UPDATE, never INSERT/DELETE). The schema enforces that with a CHECK (id = 1) constraint and seeds exactly one row.
- amount_paid on onspot_registrations is a **boolean** (“has this person paid”), not a money amount — confirmed by club/participants/page.tsx’s checkbox UI and .update({ amount_paid: !current }) toggle logic. Don’t be misled by the name.
- schools.email / schools.password_plain are real, intentional columns the app already writes to (see admin/schools/page.tsx handleCreate()) for the purpose of showing/exporting login credentials to organizers. This is discussed further in Section 6.

```
-- Extensions
create extension if not exists "pgcrypto"; -- for gen_random_uuid()

-- ----- categories -----
create table public.categories (
  id          uuid primary key default gen_random_uuid(),
```

```

    name          text not null,
    display_order integer not null default 0
);

-- ----- events -----
create table public.events (
    id              uuid primary key default gen_random_uuid(),
    category_id     uuid references public.categories(id) on delete set null,
    name            text not null,
    rulebook_url    text,
    max_entries     integer not null default 1,
    is_team_event   boolean not null default false,
    team_size       integer,
    event_date      date,
    event_time      text,                -- kept as text to match "HH:MM" string input/display as-is
    venue           text,
    assigned_members text,                -- JSON-stringified string[] – see design note above
    created_at      timestampz not null default now()
);
create index on public.events (category_id);

-- ----- schools -----
create table public.schools (
    id              uuid primary key default gen_random_uuid(),
    user_id         uuid references auth.users(id) on delete cascade,
    school_name     text not null,
    slot_number     integer unique,
    email           text,
    password_plain  text,                -- see Section 6 before going live with this as-is
    created_at      timestampz not null default now()
);

-- ----- participants -----
create table public.participants (
    id              uuid primary key default gen_random_uuid(),
    slot_number     integer not null,
    event_id        uuid not null references public.events(id) on delete cascade,
    participant_name text not null,
    entry_index     integer not null default 0,
    member_index    integer not null default 0,
    created_at      timestampz not null default now()
);
create index on public.participants (event_id);
create index on public.participants (slot_number);

-- ----- marks -----
create table public.marks (
    id              uuid primary key default gen_random_uuid(),
    slot_number     integer not null,
    event_id        uuid not null references public.events(id) on delete cascade,
    entry_index     integer not null default 0,
    scores          jsonb not null default '[]'::jsonb, -- plain array of numbers, see design note
    total           numeric not null default 0,

```

```

    finalized    boolean not null default false,
    created_at   timestampz not null default now(),
    updated_at   timestampz not null default now()
);
create index on public.marks (event_id);
create unique index marks_slot_event_entry_uq on public.marks (slot_number, event_id, entry_id);

-- ----- results -----
create table public.results (
    id            uuid primary key default gen_random_uuid(),
    event_id      uuid not null references public.events(id) on delete cascade,
    first_slot    integer,
    second_slot   integer,
    third_slot    integer,
    winners_json  text,                -- JSON-stringified WinnerGroup[] – see design note above
    published     boolean not null default false,
    created_at    timestampz not null default now(),
    updated_at    timestampz not null default now()
);
create unique index results_event_uq on public.results (event_id);

-- ----- announcements -----
create table public.announcements (
    id            uuid primary key default gen_random_uuid(),
    title         text not null,
    message       text not null,
    created_at    timestampz not null default now()
);

-- ----- onspot_registrations -----
create table public.onspot_registrations (
    id            uuid primary key default gen_random_uuid(),
    slot_number    integer not null,
    event_id      uuid not null references public.events(id) on delete cascade,
    participant_name text not null,
    amount_paid    boolean not null default false,    -- "has paid", not an amount – see note above
    created_at     timestampz not null default now()
);
create index on public.onspot_registrations (event_id);

-- ----- settings (singleton) -----
create table public.settings (
    id            integer primary key default 1 check (id = 1),
    registration_open boolean not null default true,
    points_1st    integer not null default 15,
    points_2nd    integer not null default 10,
    points_3rd    integer not null default 5
);
insert into public.settings (id, registration_open) values (1, true);

```

4. Auth strategy

Do not carry forward plaintext password storage as the authentication mechanism. Real Supabase Auth (auth.users) hashes passwords and issues JWT sessions; the app should authenticate through it, not through a hand-rolled comparison.

What changes conceptually:

- **Sign-in:** `supabase.auth.signInWithPassword({ email, password })` against real Supabase Auth — this already exists in the mock with the identical method name, so calling code in pages does not change at all.
- **Role / slot_number:** keep storing these in `user_metadata`, exactly as the mock already does (options: { data: { role, slot_number } } on sign-up). This shows up at `auth.getUser()` time as `user.user_metadata.role` — same shape the app already expects.
- **Seed users:** the five mock accounts (`admin@reflections.in / final_year`, `club@reflections.in / club_member`, `slot1/2/3@reflections.in / school`) need to be created for real, once, via the Supabase dashboard (Authentication → Users → Add user) or via a one-off script using the service-role key. Pick real passwords when you do this — don't reuse the mock's `admin123 / club123 / slot123` for anything real.

Important behavioral difference to fix, not just port over: in `admin/schools/page.tsx`, `handleCreate()` currently calls `supabase.auth.signUp()` **from the browser, while the admin is logged in**, to create a new school account. In the mock this is harmless — the mock has no real concept of “the current session.” With a **real** Supabase client, calling `auth.signUp()` client-side switches the browser's active session to the newly created user, effectively logging the admin out and logging the new school in. This is a known Supabase behavior, not a bug you'd introduce — but if you swap the client without changing this one function, every “Add School” click will silently end the admin's session.

The fix (Section 5.2) is to move account creation server-side, using the service-role key, which never touches the browser's session.

5. Code changes, file by file

5.1 `src/lib/supabase.ts` (browser client)

Replace entirely:

```
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

`@supabase/ssr` is already in `package.json` (`^0.3.0`) — no install needed.

5.2 `src/lib/supabase-server.ts` (server client)

Replace entirely with the App Router cookie-adapter pattern:

```
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export function createClient() {
  const cookieStore = cookies()
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        get(name: string) { return cookieStore.get(name)?.value },
        set(name: string, value: string, options: any) { try { cookieStore.set(name, value, options) } catch {
          remove(name: string, options: any) { try { cookieStore.set(name, '', options) } catch {
        },
      }
    }
  )
}
```

(The try/catch around set/remove is the standard Next.js App Router pattern — Server Components can't write cookies, only Route Handlers and Server Actions can, and this silently no-ops there.)

5.3 New server route for school account creation

Create `src/app/api/admin/create-school/route.ts`:

```
import { NextResponse, type NextRequest } from 'next/server'
import { createClient as createServiceClient } from '@supabase/supabase-js'

export async function POST(req: NextRequest) {
  const { email, password, school_name, slot_number } = await req.json()

  const admin = createServiceClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY! // server-only, never NEXT_PUBLIC_
  )

  const { data: authData, error: authErr } = await admin.auth.admin.createUser({
    email, password, email_confirm: true,
    user_metadata: { role: 'school', slot_number },
  })
  if (authErr) return NextResponse.json({ error: authErr.message }, { status: 400 })

  const { error: dbErr } = await admin.from('schools').insert({
    user_id: authData.user.id, school_name, slot_number, email, password_plain: password,
  })
  if (dbErr) return NextResponse.json({ error: dbErr.message }, { status: 400 })

  return NextResponse.json({ ok: true })
}
```

Then in `src/app/admin/schools/page.tsx`, change `handleCreate()` (currently around line 81-105): remove the two calls `supabase.auth.signUp(...)` and `supabase.from('schools').insert(...)`, and replace with a single call:

```
const res = await fetch('/api/admin/create-school', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password, school_name: form.school_name.trim(), slot_number: slot_number })
})
const result = await res.json()
if (result.error) { showMsg('Error creating account: ' + result.error, 'error'); setSaving(false); }
```

Nothing else in that file needs to change — `load()`, `handleDelete()`, `handleShuffle()`, the CSV/Excel/PDF export functions, and the table all read/write schools through the normal `supabase.from('schools')` builder, which works unchanged against a real client.

5.4 Everything else under `src/app` — no changes expected

These all use only `.select().eq().order().single().insert().update().upsert().delete()` against `supabase.from(table)`, which is identical between the mock and the real SDK:

`admin/dashboard`, `admin/events`, `admin/marks`, `admin/participants`, `admin/points`, `admin/results`, `admin/settings`, `admin/announcements`, `club/dashboard`, `club/participants`, `club/schedule`, `club/results`, `school/dashboard`, `school/events`, `school/events/[eventId]`, `school/results`.

One thing to double check after swapping the client, specific to `admin/events/page.tsx` (around line 79-90): it calls `supabase.storage.from('rulebooks').upload(...)` and `.getPublicUrl(...)`. The mock's storage methods are no-ops that always succeed; the real call needs the rulebooks bucket to actually exist (Section 2, step 4) and the active user's role to be allowed to write to it (Section 6 storage policy) — otherwise the upload will return a real error where the mock always returned success.

5.5 Files to delete once the migration is verified working

These exist only to support local mode and become dead code afterward:

- `src/lib/local-auth.ts`
- `src/lib/local-store.server.ts`
- `src/lib/local-engine.ts`
- `src/lib/mock-client.server.ts`
- `src/lib/mock-client.browser.ts`
- `src/lib/mock-query.ts`
- `src/lib/mock-data.ts`
- `src/app/api/local/query/route.ts`
- `src/app/api/local/auth/route.ts`
- `src/app/api/local/debug/route.ts`

Recommendation: don't delete these in the same change where you swap the Supabase clients. Keep them until you've verified the real Supabase version works end-to-end (Section 8), in case you need to fall back to local mode for a quick comparison.

6. The password_plain decision

`admin/schools/page.tsx` stores each generated school password in a `password_plain` column specifically so organizers can look it up and re-export it later (Show Passwords toggle, CSV/Excel/PDF export, per-school copy button). Real Supabase Auth never lets you read a

password back once set — there’s no API for it — so if this column didn’t exist, that whole feature would be impossible to keep, full stop. That’s almost certainly why the mock has it: it’s matching a real product requirement, not an oversight.

Storing a plaintext password column in any real database is a genuine risk if that database is ever exposed (a misconfigured RLS policy, a leaked service key, etc.). Two reasonable paths, your call:

1. **Keep password_plain, lock it down hard.** Restrict select on schools.password_plain via RLS to the final_year role only (Section 7 policy below already does this). This is the lower-effort path and matches current behavior exactly.
2. **Drop password_plain, switch the school-credentials UI to a “Reset Password” action instead of “Show Password”.** More secure, but it’s real feature work — handle-Create, downloadAllCreds/downloadSchoolCreds/Excel/PDF export, and the “Show Passwords” toggle in admin/schools/page.tsx would all need rewriting around a reset-link flow instead of a stored value.

Given the scale of this app (a single college event, schools as the only “external” accounts), option 1 is the pragmatic choice. Just don’t skip the RLS lock-down — an unrestricted password_plain column is a real exposure once you’re on a real database with a public anon key.

7. Row Level Security

Enable RLS on every table, then add policies. This is a working starting template based on the app’s three roles (final_year = full admin, club_member = operational staff, school = restricted to their own slot) — read it against your own sense of who should see what before treating it as final, since exact per-page access rules weren’t independently re-verified against every page in this pass.

```
alter table public.categories enable row level security;
alter table public.events enable row level security;
alter table public.schools enable row level security;
alter table public.participants enable row level security;
alter table public.marks enable row level security;
alter table public.results enable row level security;
alter table public.announcements enable row level security;
alter table public.onspot_registrations enable row level security;
alter table public.settings enable row level security;

-- Helper: role straight out of the JWT's user_metadata (no extra table/trigger needed)
create or replace function public.current_role() returns text as $$
  select coalesce(auth.jwt() -> 'user_metadata' -> 'role', '')
$$ language sql stable;

create or replace function public.current_slot() returns integer as $$
  select (auth.jwt() -> 'user_metadata' -> 'slot_number')::integer
$$ language sql stable;

-- categories / events / announcements / results / settings: everyone signed-in can read,
-- only final_year (and club_member where the app lets them edit) can write.
create policy "read all - categories" on public.categories for select using (auth.role() = 'auth')
create policy "write admin - categories" on public.categories for all using (public.current_role() = 'final_year')
```

```

create policy "read all - events" on public.events for select using (auth.role() = 'authenticated');
create policy "write admin+club - events" on public.events for all using (public.current_role() = 'admin' or public.current_role() = 'club_member');

create policy "read all - announcements" on public.announcements for select using (auth.role() = 'authenticated');
create policy "write admin+club - announcements" on public.announcements for all using (public.current_role() = 'admin' or public.current_role() = 'club_member');

create policy "read all - results" on public.results for select using (auth.role() = 'authenticated');
create policy "write admin+club - results" on public.results for all using (public.current_role() = 'admin' or public.current_role() = 'club_member');

create policy "read all - settings" on public.settings for select using (auth.role() = 'authenticated');
create policy "write admin - settings" on public.settings for update using (public.current_role() = 'admin');

-- schools: final_year sees/edits everything; a school can read its own row but not others',
-- and never password_plain (enforced via a column-level revoke, not RLS).
create policy "admin full access - schools" on public.schools for all using (public.current_role() = 'admin');
create policy "school reads own row" on public.schools for select using (
    public.current_role() = 'school' and slot_number = public.current_slot()
);
revoke select (password_plain) on public.schools from authenticated;
grant select (id, user_id, school_name, slot_number, email, created_at) on public.schools to authenticated;

-- participants / marks: club & admin see everything; a school only sees/edits its own slot.
create policy "admin+club full access - participants" on public.participants for all using (public.current_role() = 'admin' or public.current_role() = 'club_member');
create policy "school own slot - participants" on public.participants for all using (
    public.current_role() = 'school' and slot_number = public.current_slot()
);

create policy "admin+club full access - marks" on public.marks for all using (public.current_role() = 'admin' or public.current_role() = 'club_member');
create policy "school reads own slot - marks" on public.marks for select using (
    public.current_role() = 'school' and slot_number = public.current_slot()
);

-- onspot_registrations: club & admin manage these; schools have no access (on-spot is club-run)
create policy "admin+club full access - onspot" on public.onspot_registrations for all using (public.current_role() = 'admin' or public.current_role() = 'club_member');

```

8. Storage policy for the rulebooks bucket

```

-- Public read (rulebook PDFs are meant to be visible to all participants)
create policy "public read rulebooks" on storage.objects for select using (bucket_id = 'rulebooks');
-- Only admin/club can upload
create policy "admin+club upload rulebooks" on storage.objects for insert with check (
    bucket_id = 'rulebooks' and public.current_role() in ('final_year', 'club_member')
);

```

9. Environment variables

.env.local today:

- **events:** id, category_id, name, rulebook_url, max_entries, is_team_event, team_size, event_date, event_time, venue, assigned_members, created_at — joined as events.categories via category_id wherever events are fetched with .select('*', categories(name, display_order)').
- **schools:** id, user_id, school_name, slot_number, email, password_plain, created_at
- **participants:** id, slot_number, event_id, participant_name, entry_index, member_index, created_at — joined as participants.events via event_id.
- **marks:** id, slot_number, event_id, entry_index, scores (array), total, finalized, created_at, updated_at
- **results:** id, event_id, first_slot, second_slot, third_slot, winners_json (JSON string), published, created_at, updated_at — joined as results.events (which itself nests .categories) via event_id.
- **announcements:** id, title, message, created_at
- **onspot_registrations:** id, slot_number, event_id, participant_name, amount_paid (boolean), created_at
- **settings:** singleton row, id=1, registration_open, points_1st, points_2nd, points_3rd
- **auth users** (not a custom table — real auth.users + user_metadata): {role: 'school' | 'club_member' | 'final_year', slot_number?}